

Python More Image Than Memory (MITM) Viewer Software User Manual

1. Introduction

1.1 Purpose

The More Image Than Memory (MITM) application is a specialized software tool designed for displaying and performing simple processing operations on complex Synthetic Aperture Radar (SAR) data using the NGA Sensor Independent Complex Data (SICD) format.

1.2 Document Overview

This Software User Manual (SUM) is structured to provide users with all the necessary information to install and effectively use the MITM software. The manual is organized as follows:

1. **Introduction:** An overview of the software's purpose and design
2. **Installation Instructions:** Step-by-step guidance for setting up the software
3. **Graphical User Interface:** Detailed explanation of the interface and navigation
4. **Model View Controller:** Technical overview of the software architecture
5. **Conclusion:** Documentation summary

This manual assumes users have technical familiarity with SAR imagery and its applications but may not have extensive software development experience.

1.3 System Design

MITM is architected to efficiently handle large, complex SAR datasets while providing an intuitive interface for technical users. Several key design decisions inform the system architecture:

1.2.1 Model-View-Controller (MVC) Architecture

The software employs the MVC architectural pattern, offering several advantages:

- Clear separation of concerns, making the codebase more maintainable
- Modular design allowing independent development of components
- Simplified adaptation to changing requirements

1.2.2 PySide Graphics Library

MITM leverages the PySide (Qt for Python) framework for its graphical interface, which provides:

- Cross-platform compatibility (Windows, Linux)
- Native look and feel on each system
- Extensive widget library for specialized data displays
- Modern, customizable UI components

PySide was chosen over PyQt for its licensing and other advantages:

- LGPL license allows for both open-source and commercial use without requiring disclosure of application code
- No commercial licensing fees required for deployment
- Same Qt functionality and API as PyQt
- Growing community support, with more intuitive documentation

1.2.3 PyQtGraph Integration

MITM utilizes the PyQTGraph library for specialized image display capabilities:

- High-performance display of scientific/engineering image data
- Hardware-accelerated rendering for smooth interaction with large datasets
- Built-in support for complex image transformations and color mapping
- Interactive region-of-interest selection and analysis
- Seamless integration with the PySide framework
- Optimized for real-time updates and interactive visualization of SAR data

1.2.4 Memory Management Strategy

To handle SAR datasets that often exceed available system memory, MITM implements:

- Chunked data access patterns for efficient reading of large files
- Decimation strategy for image viewing that dynamically reduces resolution based on current zoom level and viewport size
- Intelligent disposal of out-of-view data chunks to minimize memory footprint

1.2.5 Concurrency Model

MITM employs a robust concurrent model based on Qt's QThread framework:

- Background processing for computationally intensive operations
- Responsive UI maintained during complex calculations
- Worker threads for data loading and processing operations
- Thread-safe communication via Qt's signals and slots mechanism

2. Installation Instructions

2.1 GitLab Download and Setup

If you are downloading MITM from a remote repository, follow the below steps to begin running MITM:

1. Navigate to the directory you would like to clone MITM into
2. `$ git clone <"Clone with HTTPS/SSH" URL>`
3. (Optional) Create a Python virtual environment (venv, conda, etc.) (Python 3.11 or greater)
4. Install the required packages by running: `$ pip install -r requirements.txt`
5. Run MITM: `$ python main.py`

2.2 Packaging MITM for Distribution

MITM is packaged using PyInstaller, which bundles Python, MITM, and all of its dependencies into a single executable which may then be distributed to other machines. The resulting executable is fully self-contained.

PyInstaller requires that MITM be packaged on the same system as the machines the resulting executable will be distributed to. For example, to run the executable on a Windows 10 machine, it must have been created on a Windows 10 machine, etc.

2.2.1 Packaging MITM as a Stand-Alone Application

How to package MITM as a stand-alone executable with PyInstaller:

1. pip install python-mitm (if it is not installed)
2. Navigate to the root of the MITM repository
3. \$ cd standalone/pyinstaller
4. \$ pyinstaller mitm.spec

How to run the resulting executable:

- \$./dist/mitm.exe

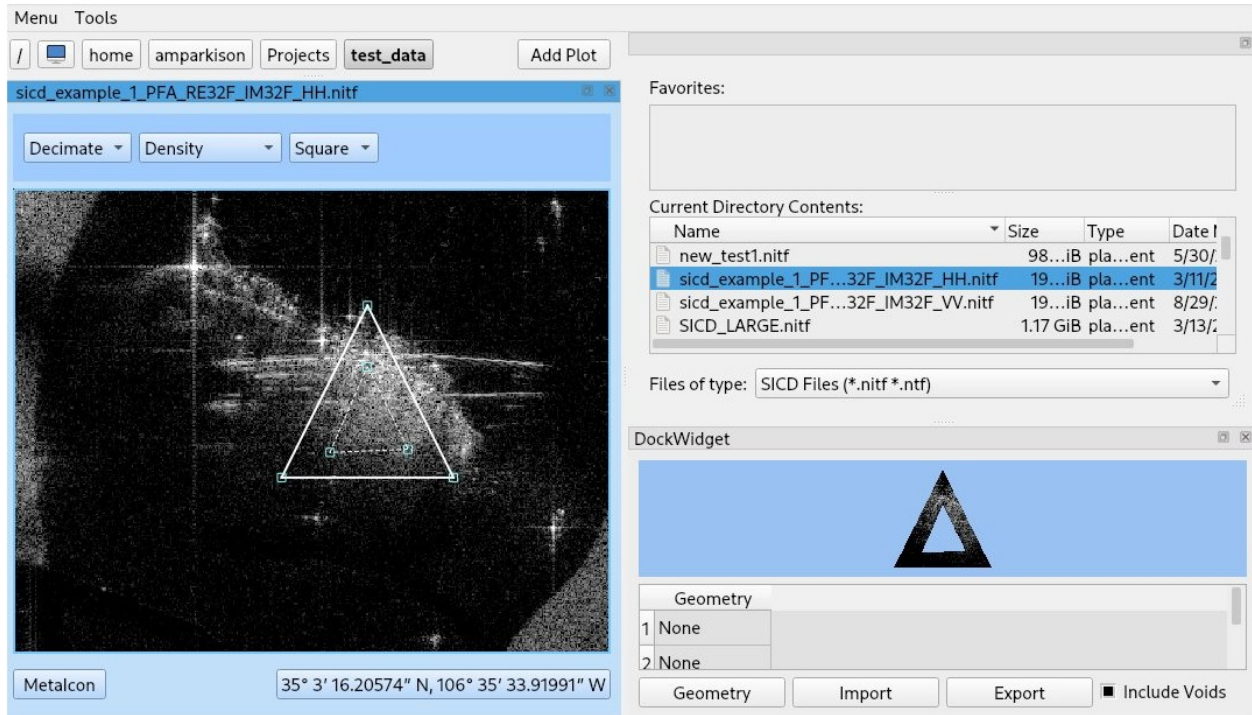
2.2.2 Packaging MITM with Sub-Applications

Packaging MITM with sub-applications (e.g. RCS Tool) requires modifications to mitm.spec. These changes are outlined below. For best results, installing all sub-applications with pip is recommended. Variable names within mitm.spec are in bold.

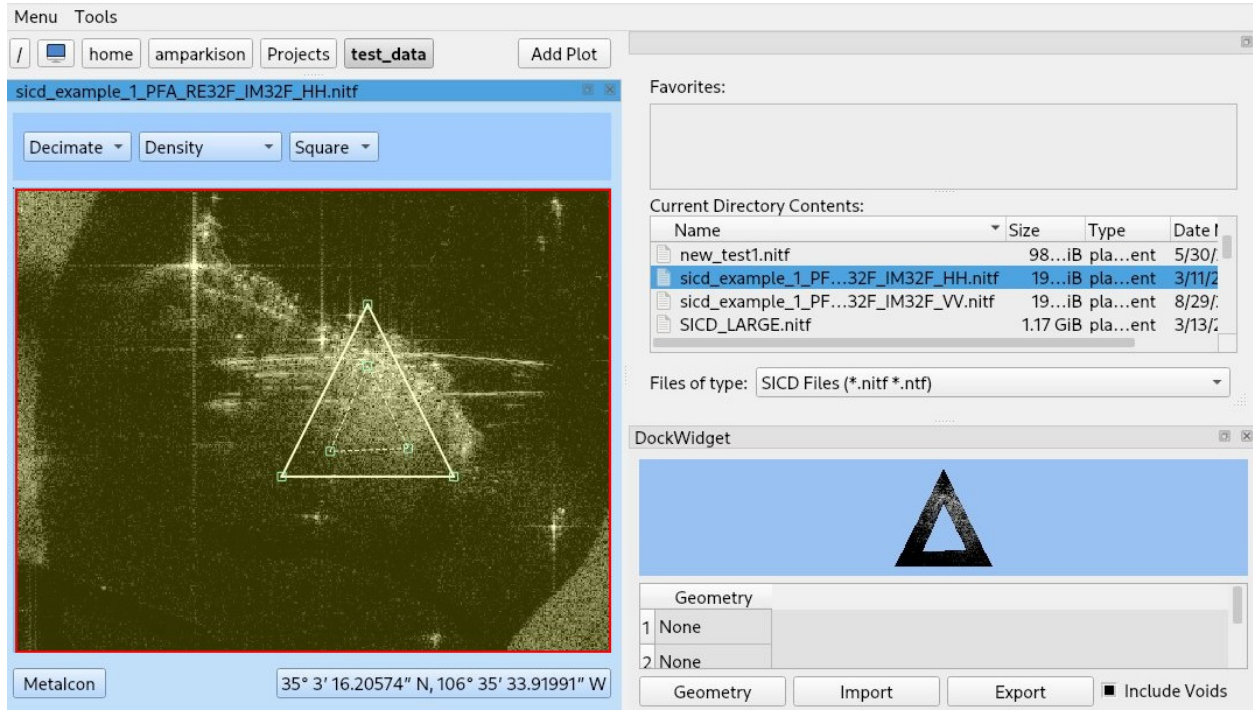
1. Any non-python files the app relies on should be added to **data_files** along with the specific module that relies upon them.
2. The first argument to the **Analysis** object should be the path to the runner file (typically main.py). This path may be relative to mitm.spec or absolute.
3. Add the module name of the sub-application to the list passed to **hiddenimports**. PyInstaller sometimes has difficulty finding modules and adding them here does not hurt.
4. (Optional) In the constructor for the **EXE** object, the **name** argument may be modified to change the name of the created executable.

3. Graphical User Interface (GUI)

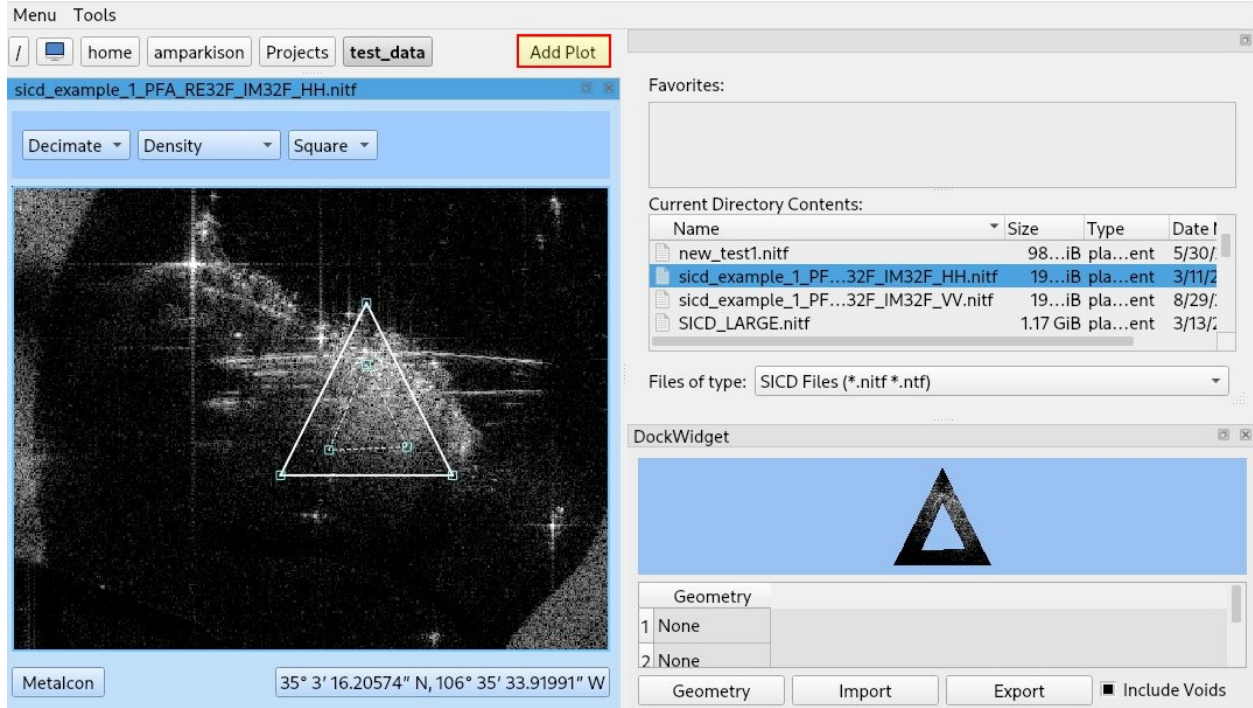
3.1 Main Interface Overview



The MITM graphical user interface is designed to provide intuitive access to complex SAR data visualization capabilities.

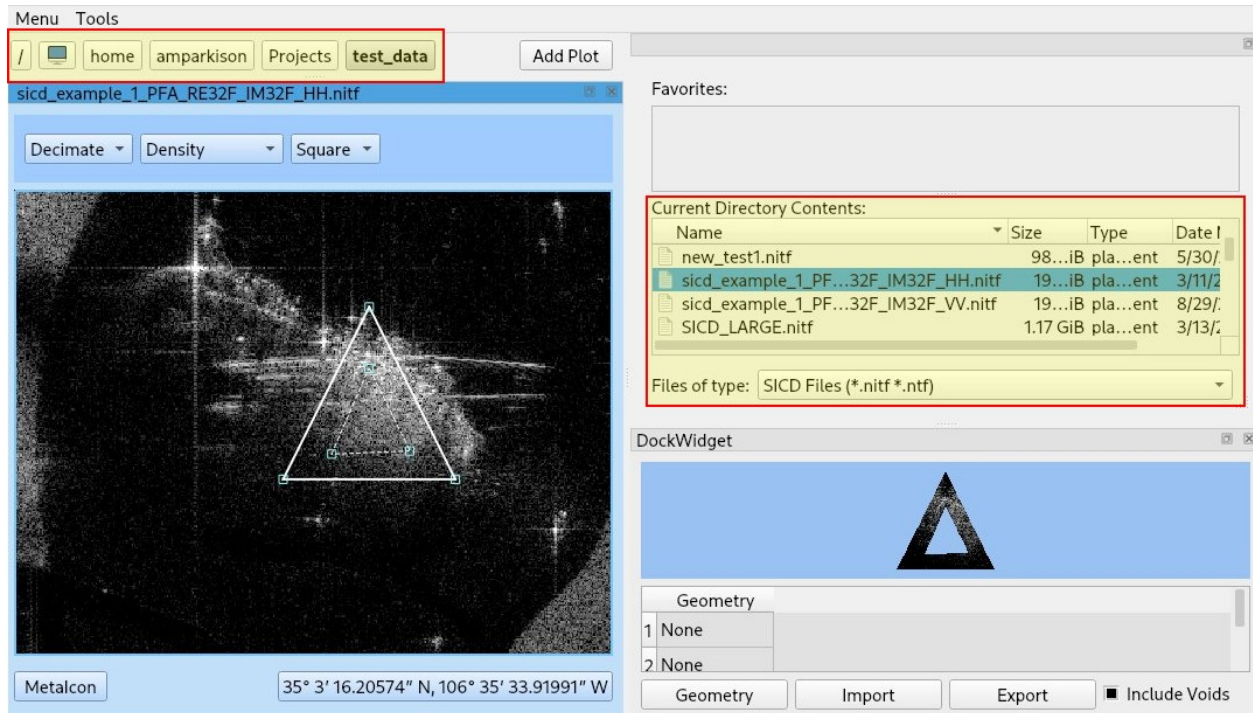


This main display window on the left is where the user can explore the imported SAR image. The user can zoom in/out (the decimation will automatically update) as well as pan around. The user can dock/un-dock this view and place it wherever.



The user can add as many of these plots as they wish by clicking the **"Add Plot"** button or double clicking a file in the **"Current Directory Contents"** list.

Add Plot: Add another plot window that displays SAR data and geometries.

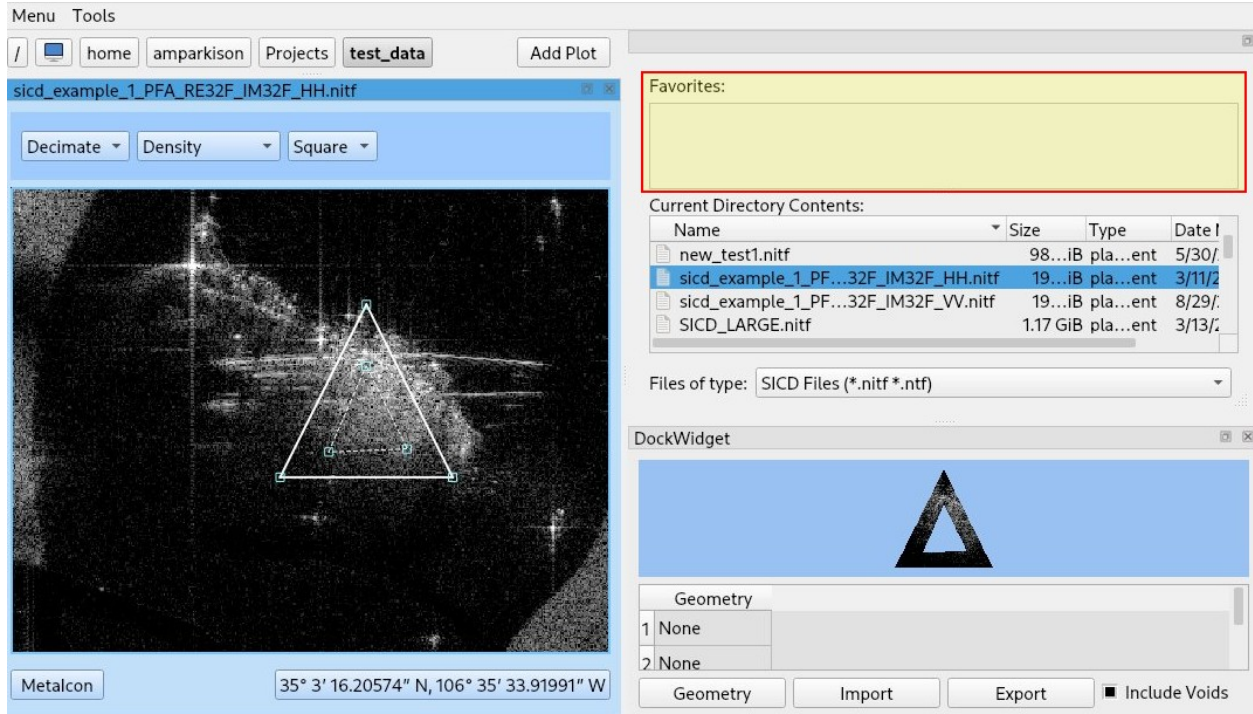


In the top left of MITM viewer we have the **"Directory Buttons"**, this allows the user to navigate between different directories. The current file contents (filtered dependent on the **"Files Of Type"** combo box) is displayed in the **"Current Directory Contents"** list.

Directory Buttons: Populated array of buttons, where each button represents a directory on the user computer. Used for navigating between directories.

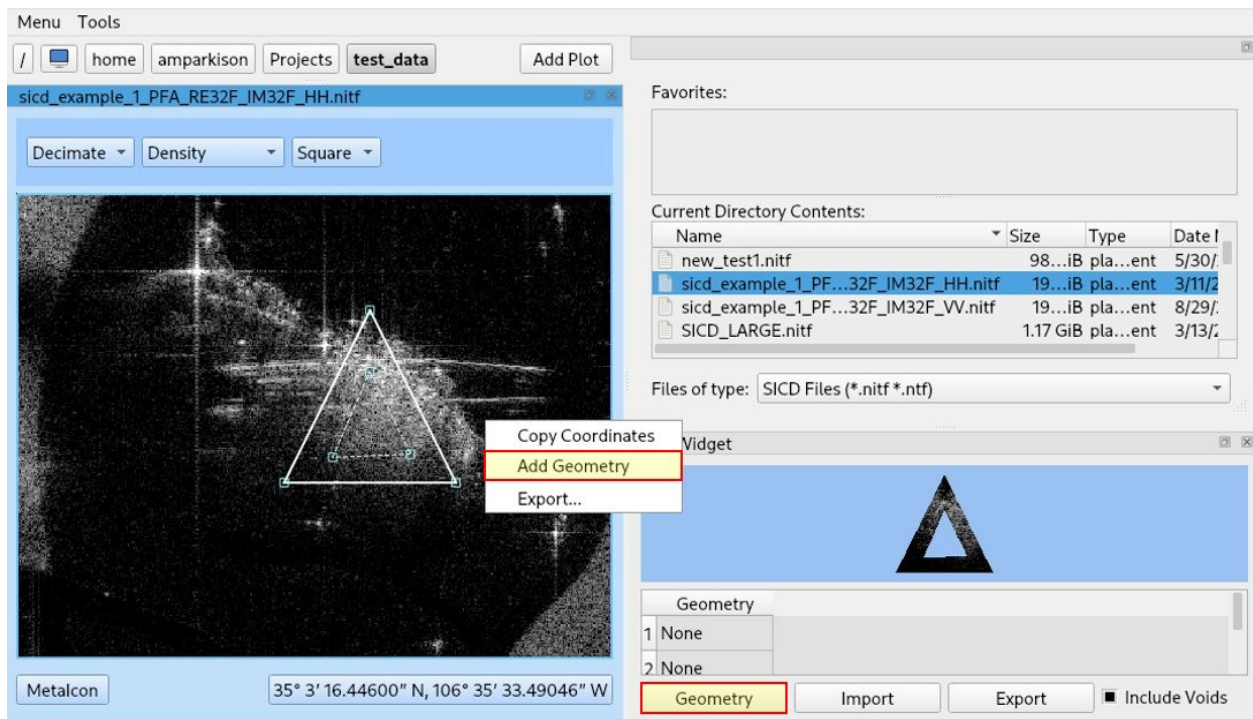
Files Of Type: Combo box allowing the user to select which SAR file to filter by.

Current Directory Contents: Displays a list of SAR files in the current directory, users can double click to add a new tabbed plot of this data or drag and drop the file into a plot.



Above "Current Directory Contents" list, there is a "Favorites" list.

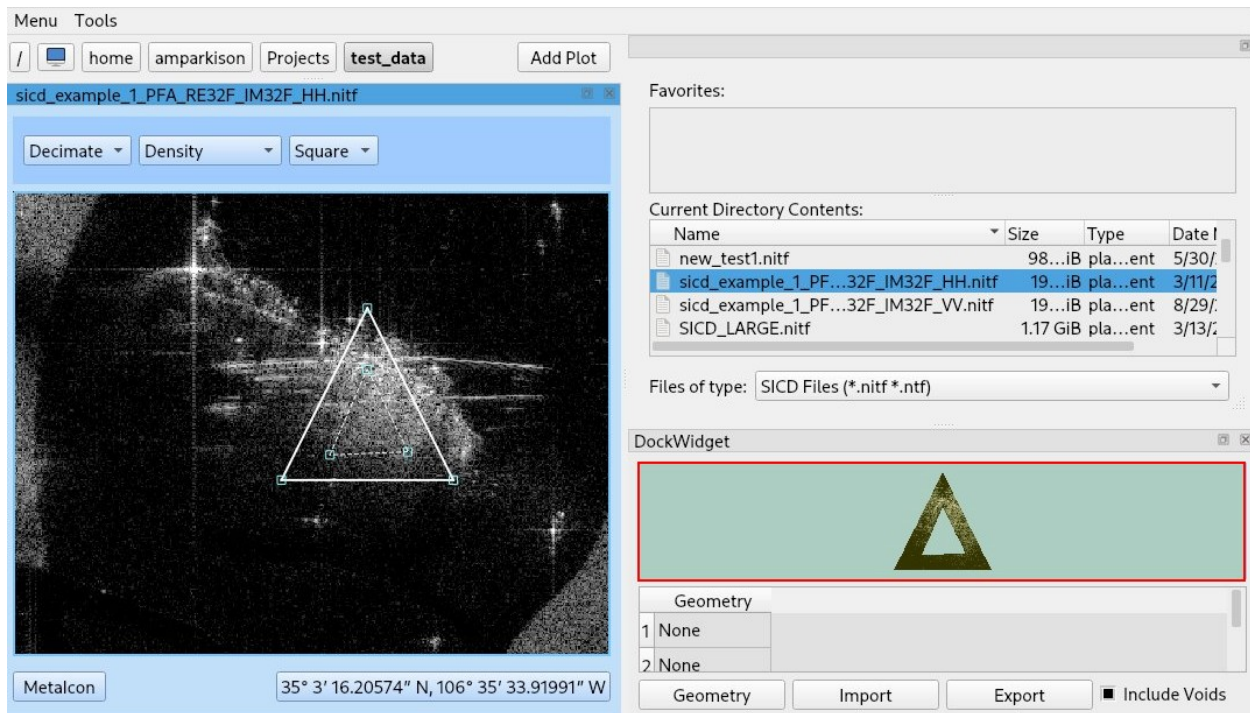
Favorites: Allows the user to type/drag a favorite directory or network drive to be opened. This is saved to a config file that get's loaded into MITM when you restart the application.



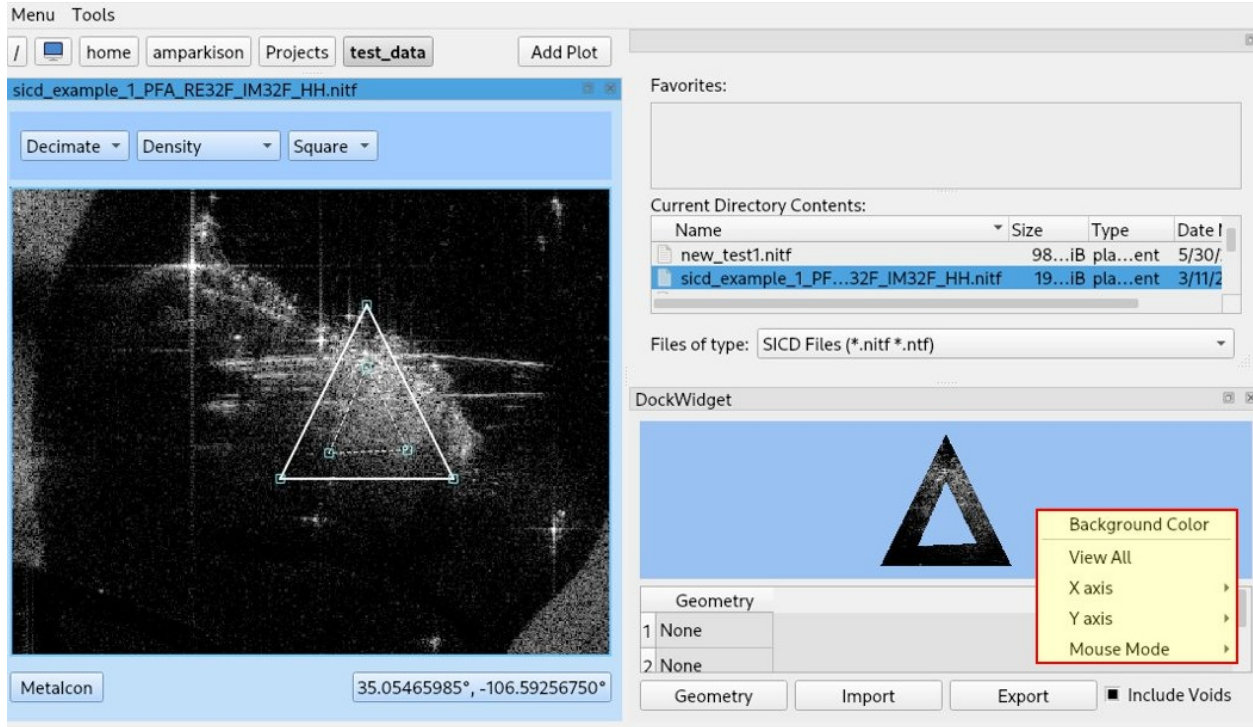
Users can add geometries to their currently selected plot by either clicking the **“Geometry”** button in the bottom right corner of the screen or right clicking a plot and clicking **“Add Geometry”**.

Geometry: Adds a new geometry to the currently selected plot. This geometry defaults to a triangle geometry spawned in the center of the current field of view of the plot and scaled based on the current zoom level.

Add Geometry: Right clicking the plot and selecting **“Add Geometry”** spawns a new geometry at the users current cursor position. This geometry defaults to a triangle geometry scaled based on the current zoom level.



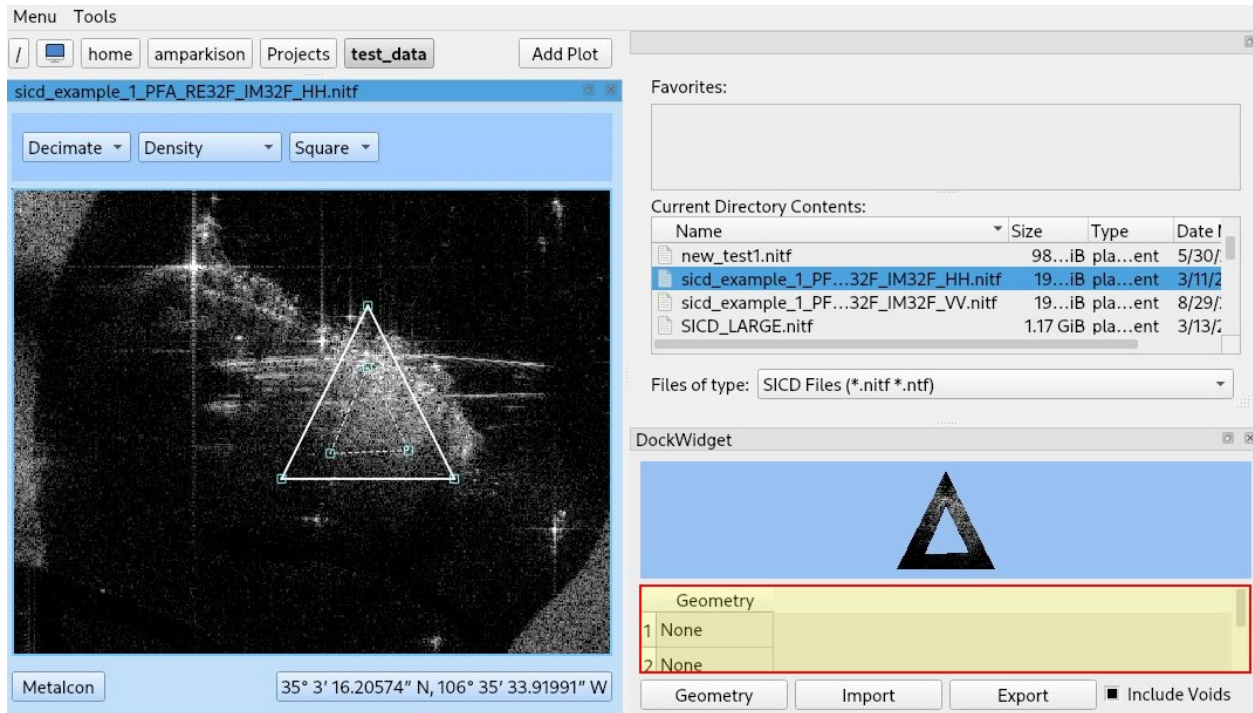
On the right side of the GUI there is a **“Geometry Preview”** display. This displays the currently selected geometry and depending on if **“Include Voids”** is toggled, any void in the interior of the selected geometry. This preview aids the user in ensuring the selection that was made is accurate to what they are trying to select.



Right clicking the **"Geometry Preview"** gives the **"Background Color"** option.

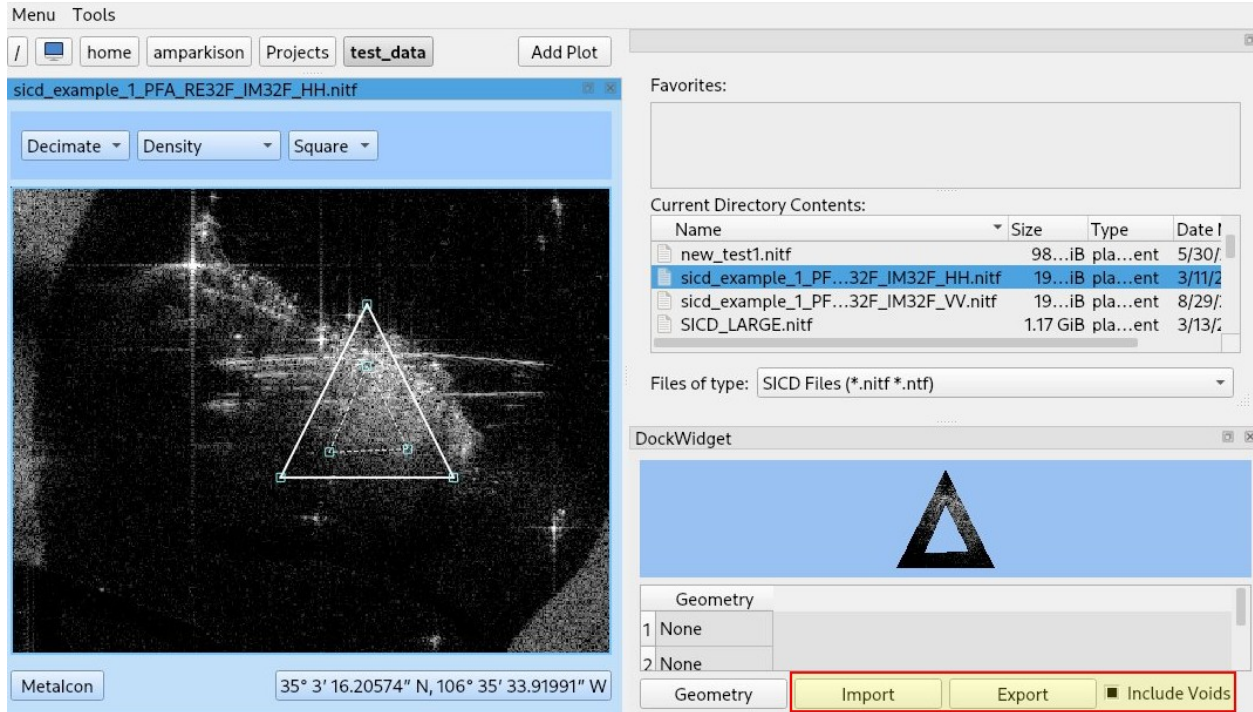
Geometry Preview: Displays a preview of only the currently selected geometry.

Background Color: Allows the user to change the background color of the **"Geometry Preview"**.



Below the “**Geometry Preview**” display is the “**Geometry Table**”.

Geometry Table: A table that is populated with all the geometries that are on the currently selected plot. Allows the user to name geometries (saved to .json when exported) and interact with the geometries through the table (selecting between different geometries).

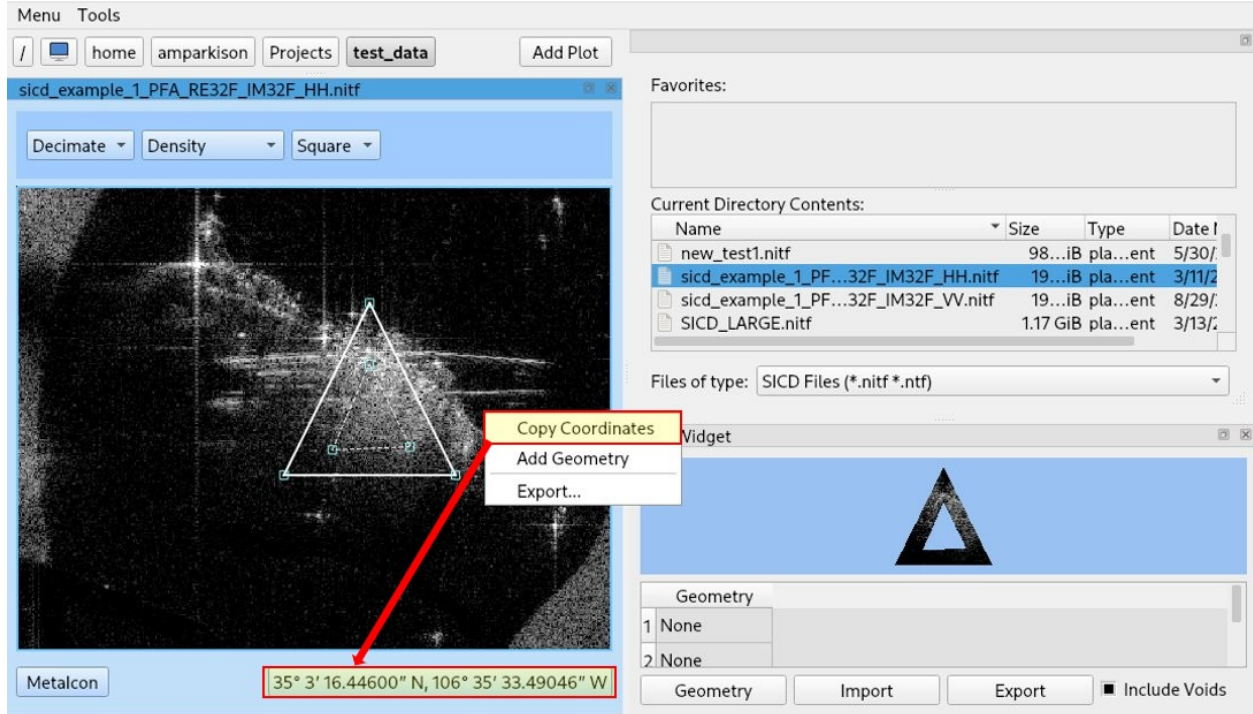


The bottom right hand corner of the GUI has options relating to importing and exporting.

Import: Allows the user to import one or multiple geometries. The user can import either from a .json (following geo-json standard) or a .mat (supporting legacy matlab tools).

Export: Allows the user to export all geometries on the currently selected widget. The geometries are saved as a .json (followin geo-json standard).

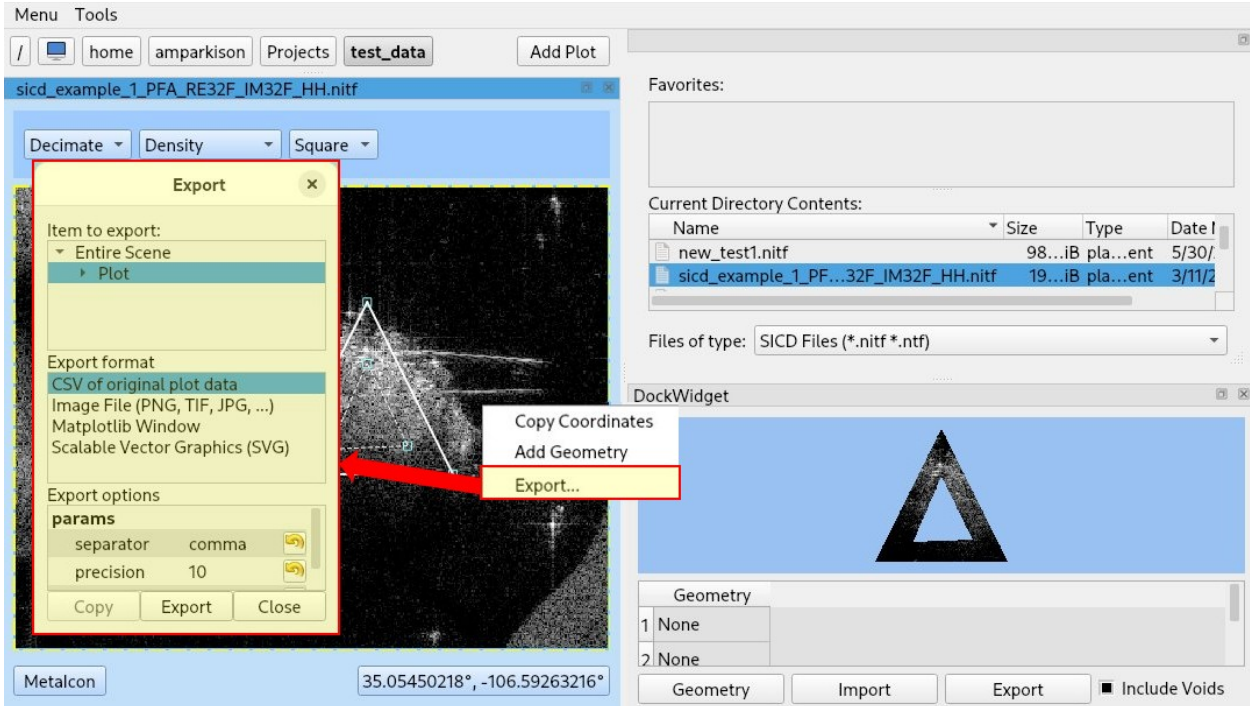
Include Voids: Check-box for toggling between including and excluding voids. This will modify what is shown in the "**Preview Geometry**" as well as how the geometries on the currently selected plot are exported with the "**Export**" button".



Right clicking the plot also gives the user options to **"Copy Coordinates"** which copies coordinates shown in the **"Coordinate Display"**.

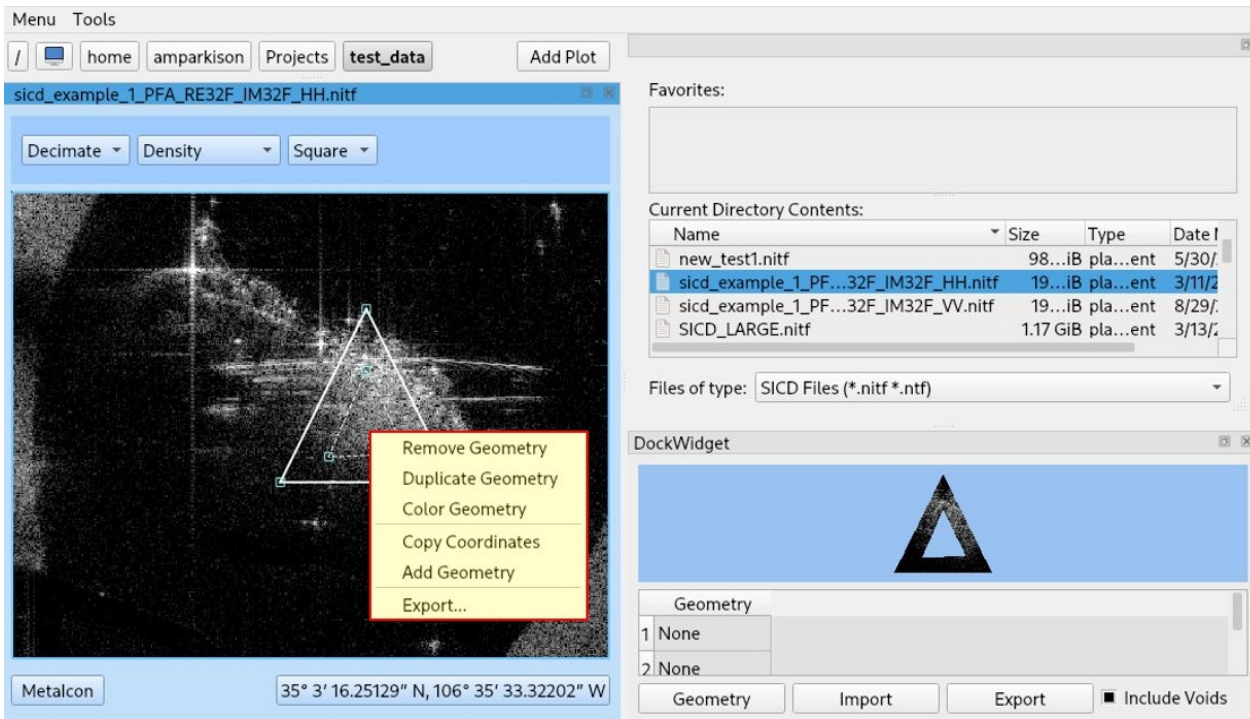
Copy Coordinates: Users can copy the currently displayed coordinates from the **"Coordinate Display"** shown in the bottom right corner of each plot. These can then be pasted into other software.

Coordinate Display: Displays the coordinates related to the users current mouse position on a selected plot. The user can click this to switch between different display formats for the coordinates.



Right clicking the plot and clicking “**Export**” allows the user to export the plot.

Export: The user can export the currently selected plot as a CSV (complex data), Image File, Matplotlib window, or SVG.

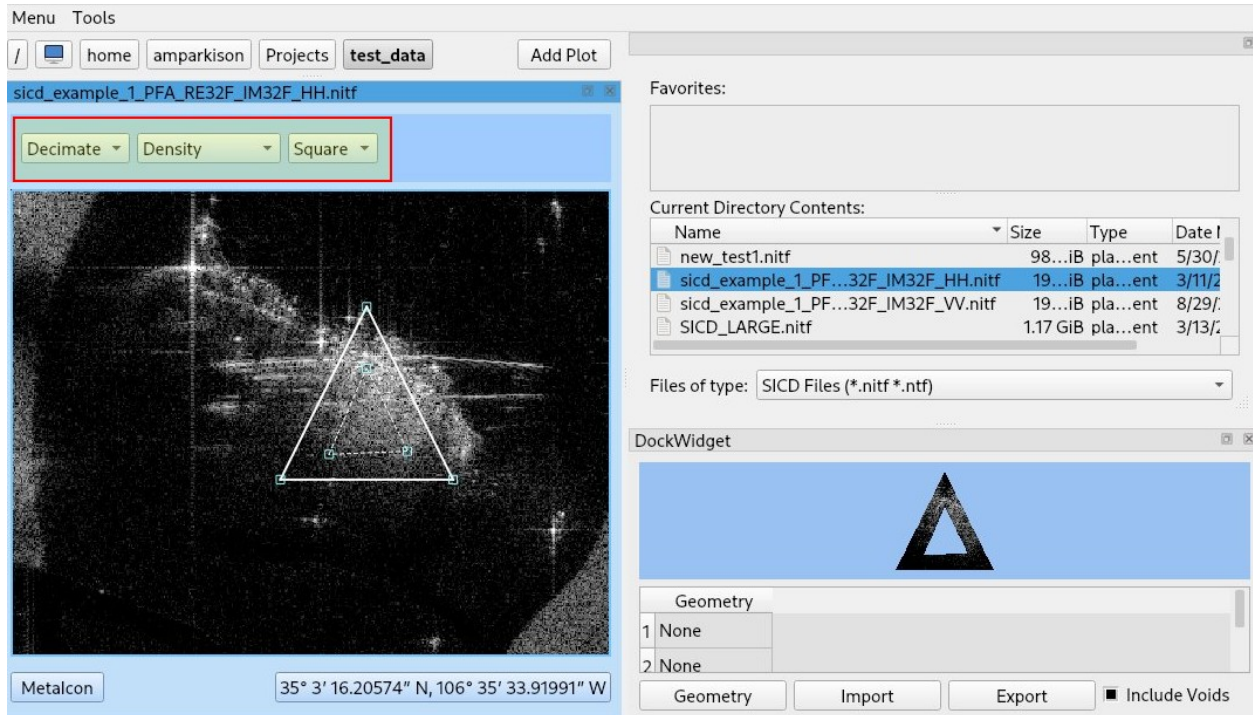


Right clicking a specific geometry on a plot gives the user additional options relating to that geometry: “**Remove Geometry**”, “**Duplicate Geometry**”, and “**Color Geometry**”.

Remove Geometry: Removes the currently selected geometry.

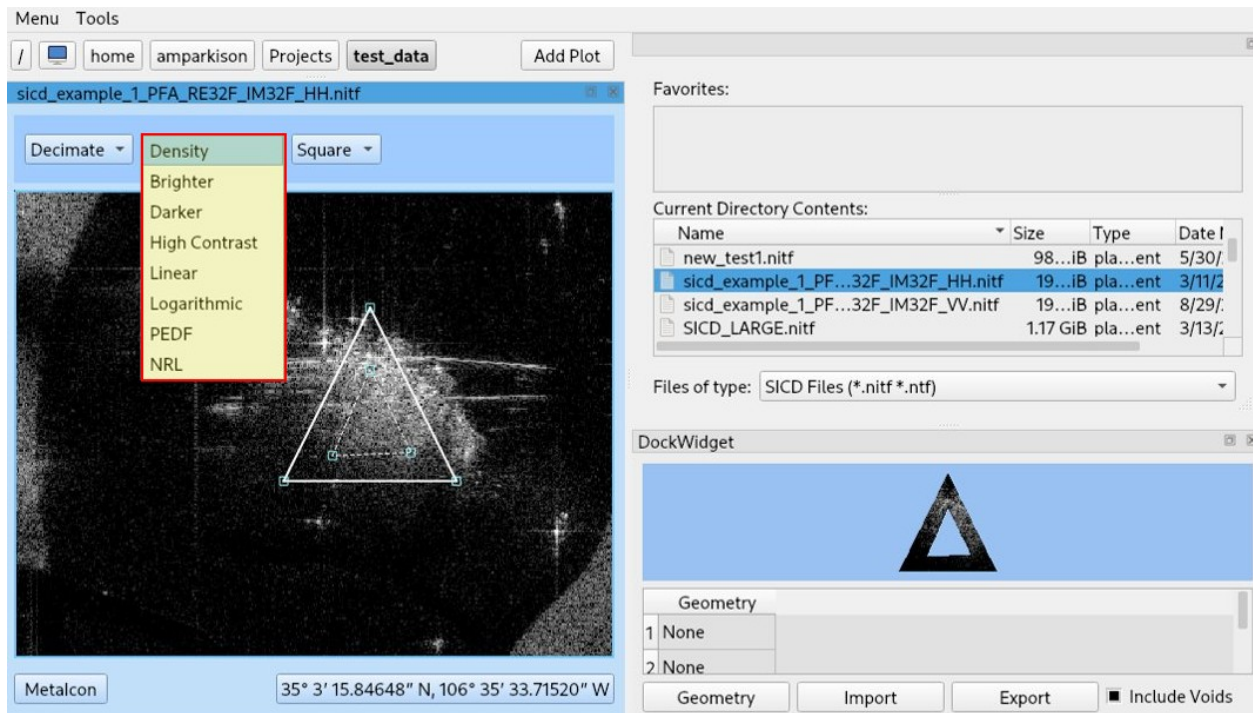
Duplicate Geometry: Duplicates the currently selected geometry, spawning directly on top of original.

Color Geometry: Opens up a dialog box allowing the user to change the color of the geometry. This also updates the json to save this color when exporting.

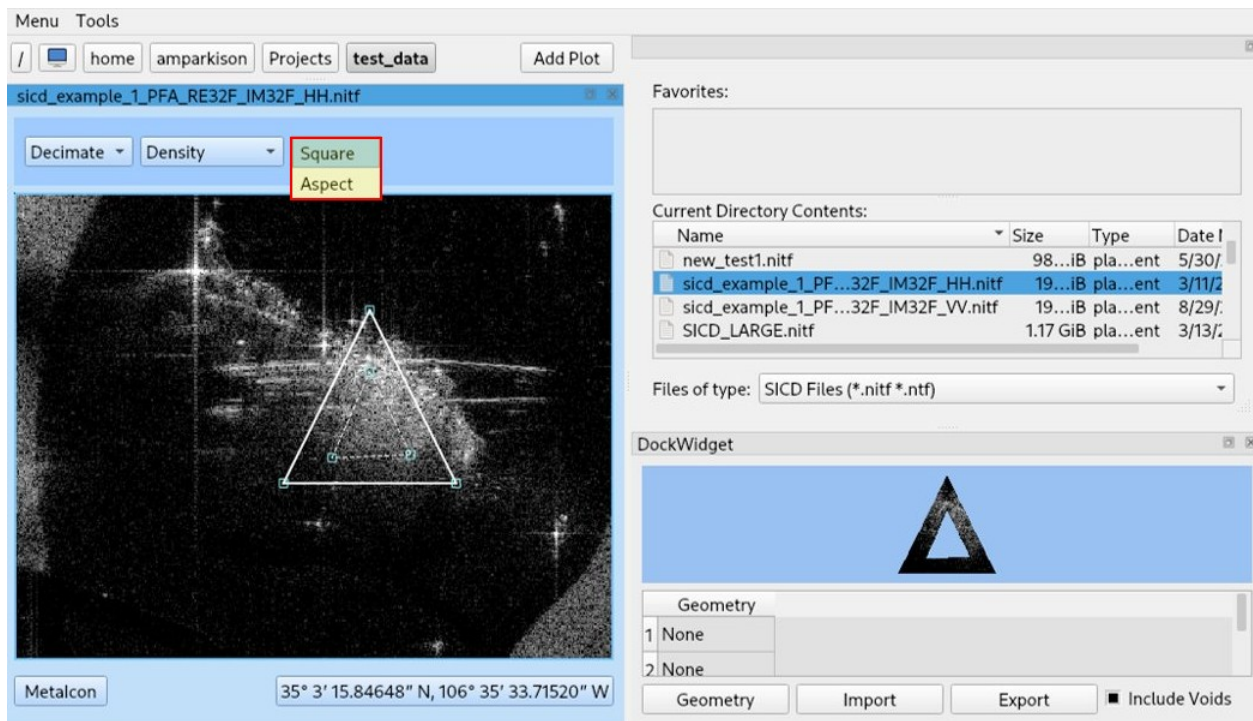


The top of each plot has combo boxes for allowing the user to select settings applied to the SAR image: **"Decimation Method"**, **"Remapping"**, **"Pixel Shape"** (going from left to right).

Decimation Method: Allows the user to select between different decimation methods, currently the only decimation method is nth pixel method.

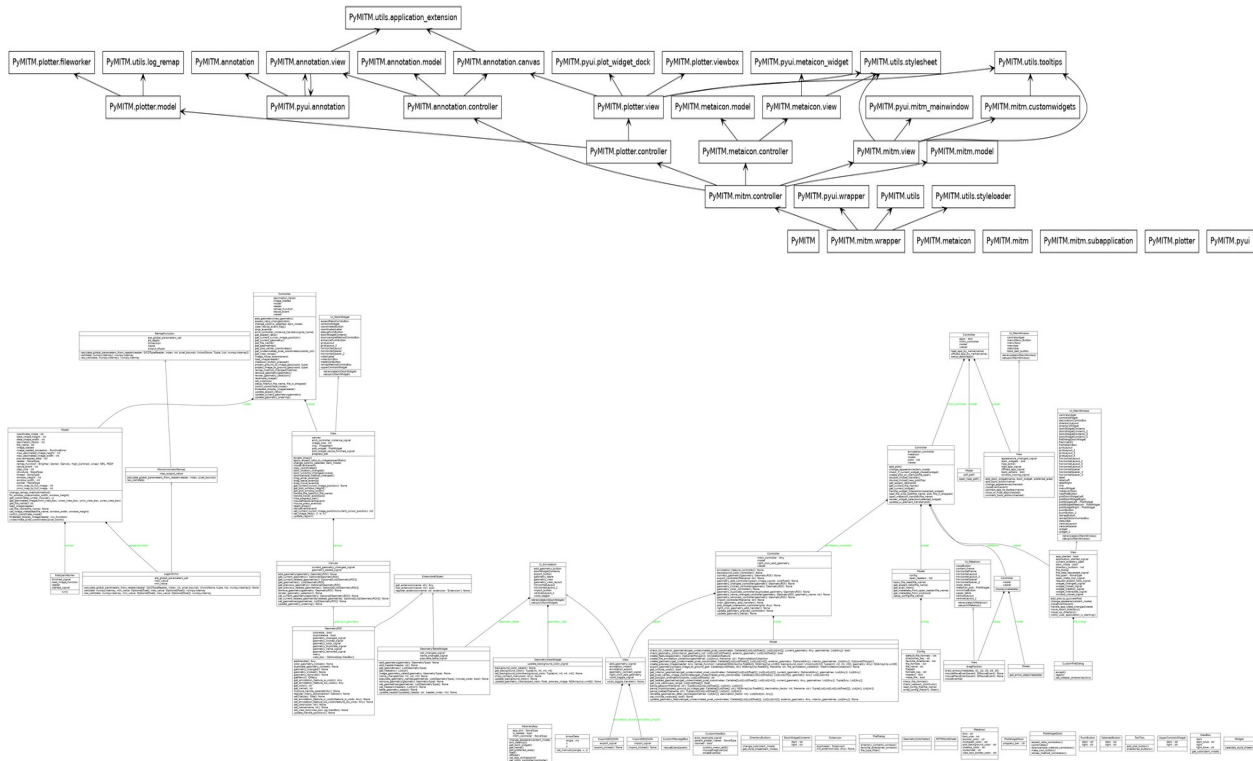


Remapping: Allows the user to select between different linear re-mappings for displaying the data.



Pixel Shape: Allows the user to select between square and aspect ratio matching pixels.

4. MVC



5. Conclusion

The More Image Than Memory (MITM) Viewer provides practical capabilities for working with complex SAR data. By addressing the basic challenge of handling datasets larger than available memory, MITM allows users familiar with SAR imagery to view and analyze SICD-formatted data.

Key aspects of MITM include:

1. **Efficient Memory Usage:** Chunked data access and dynamic decimation strategies help manage large SAR datasets on systems with limited resources.
2. **Practical User Interface:** The modular GUI with customizable plot widgets lets users organize their workspace as needed.
3. **Solid Technical Design:** The Model-View-Controller architecture paired with PySide and PyQtGraph libraries provides a stable foundation.
4. **Multiple Visualization Options:** Different resampling and remapping options help users adjust their view of SAR data as needed.

SAR datasets continue to grow in size and complexity, making tools that can handle large files increasingly necessary. MITM aims to provide a straightforward solution for this challenge.

For questions or additional information, please refer to the contact details provided in section 6.